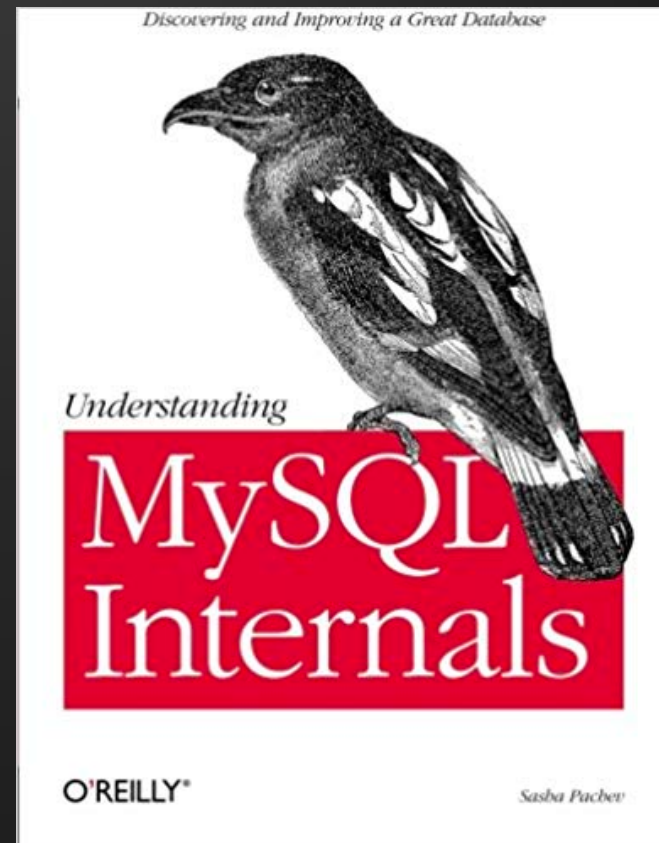
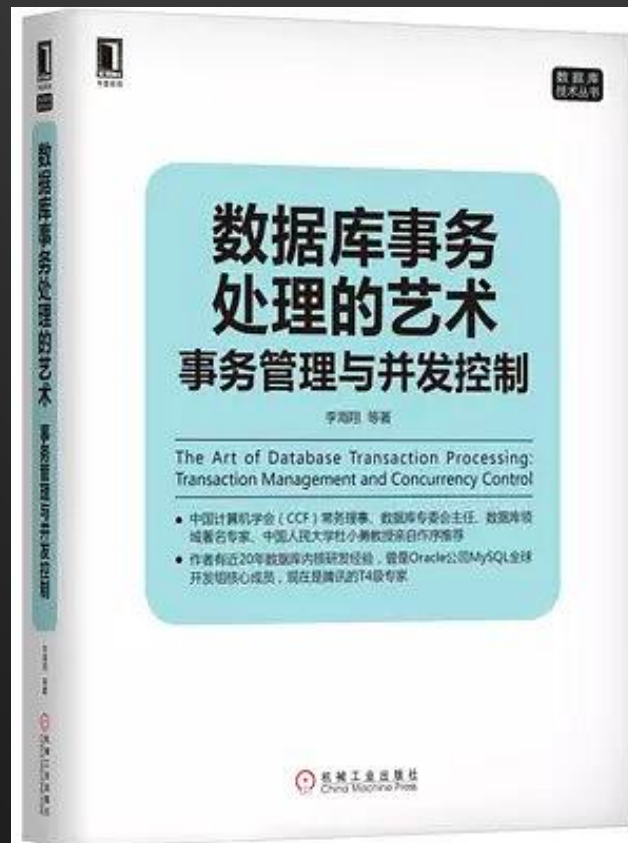
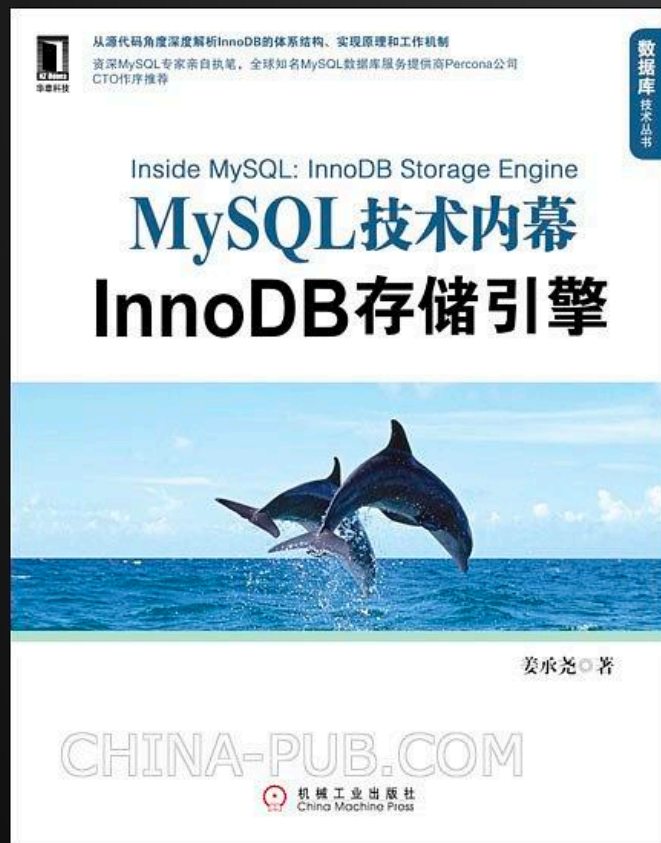
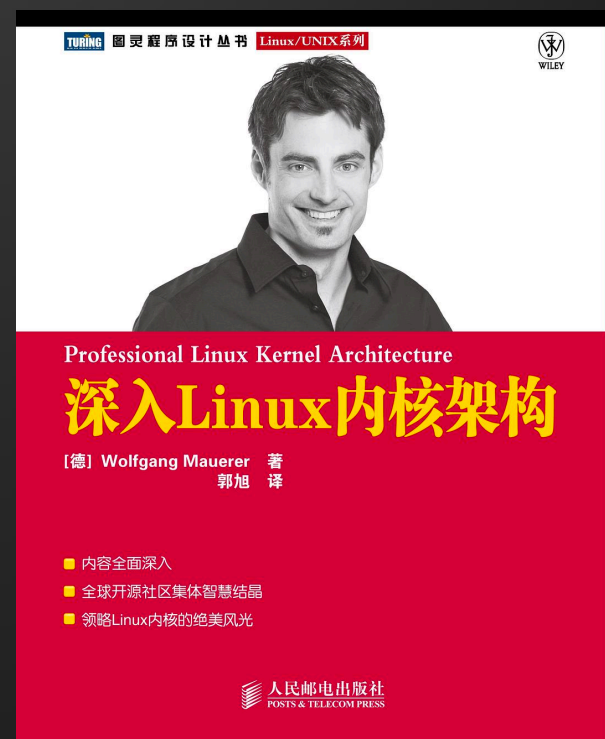
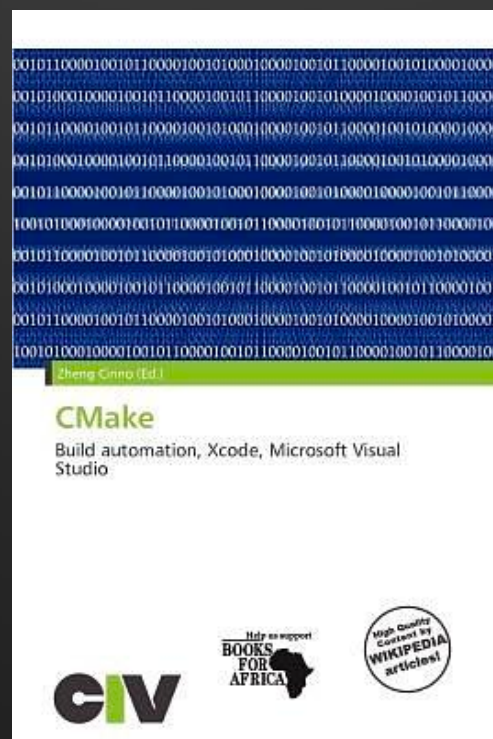
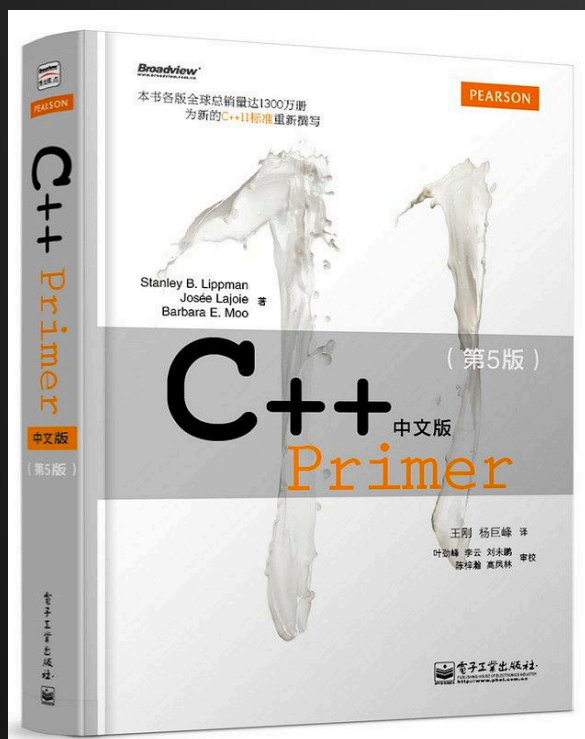


如何"土土"地 读MySQL源码

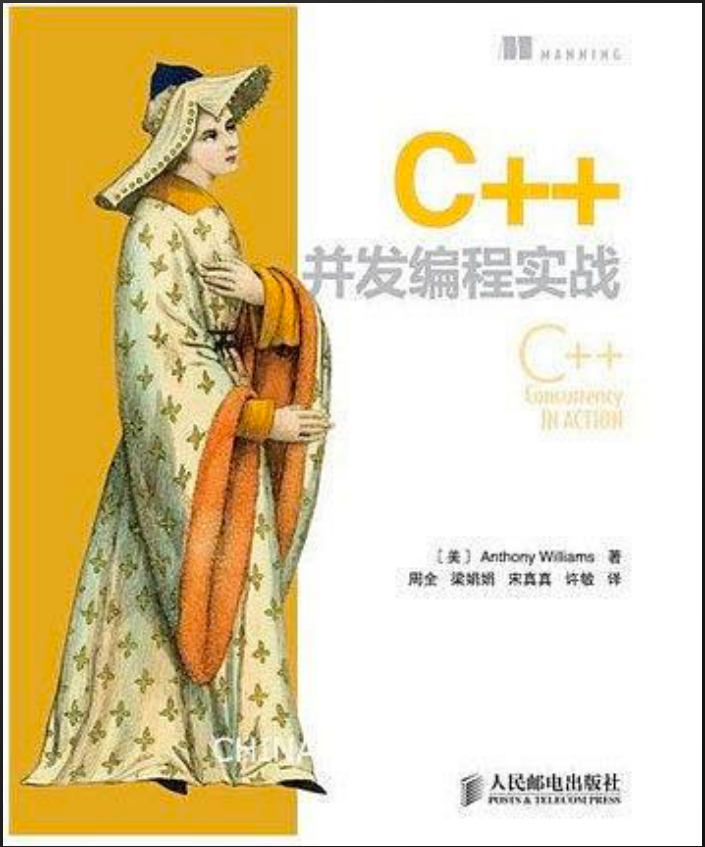
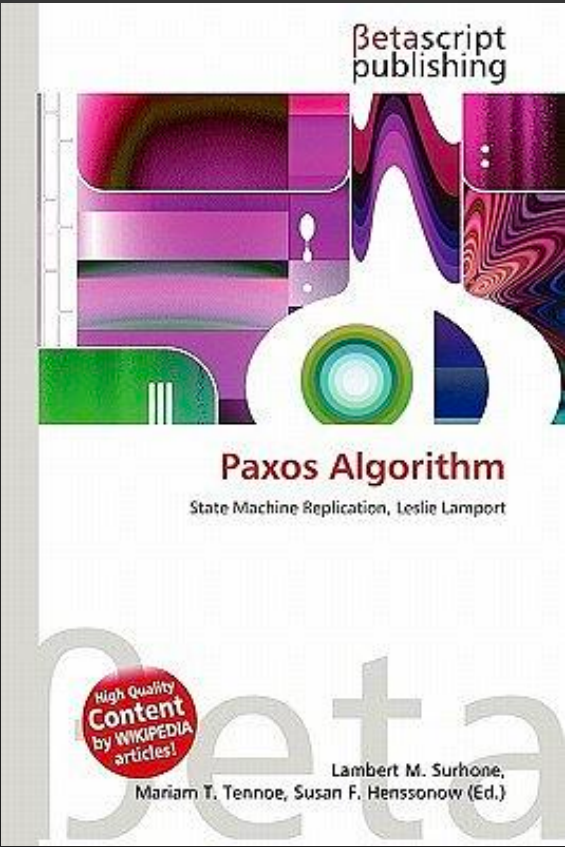
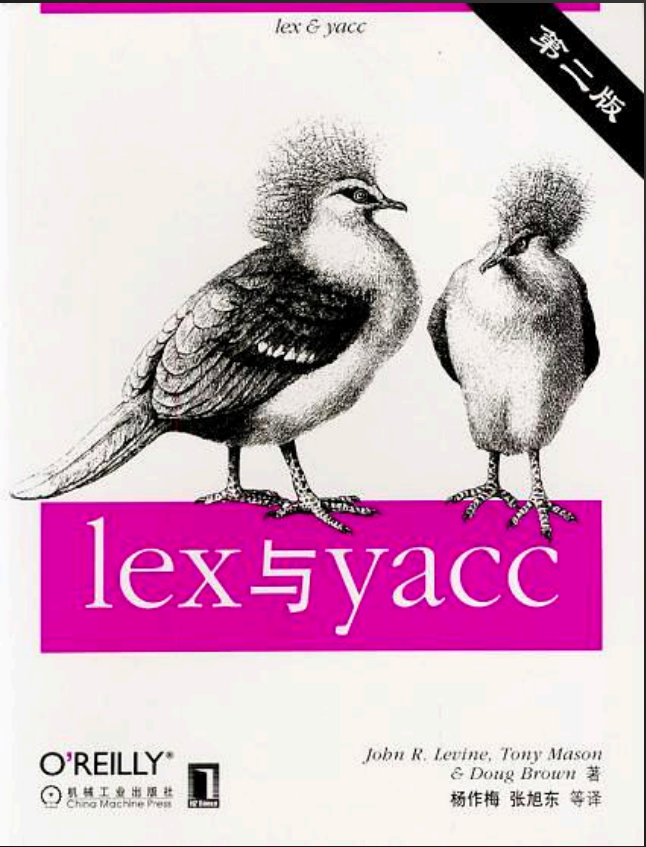
为什么要"土土"地



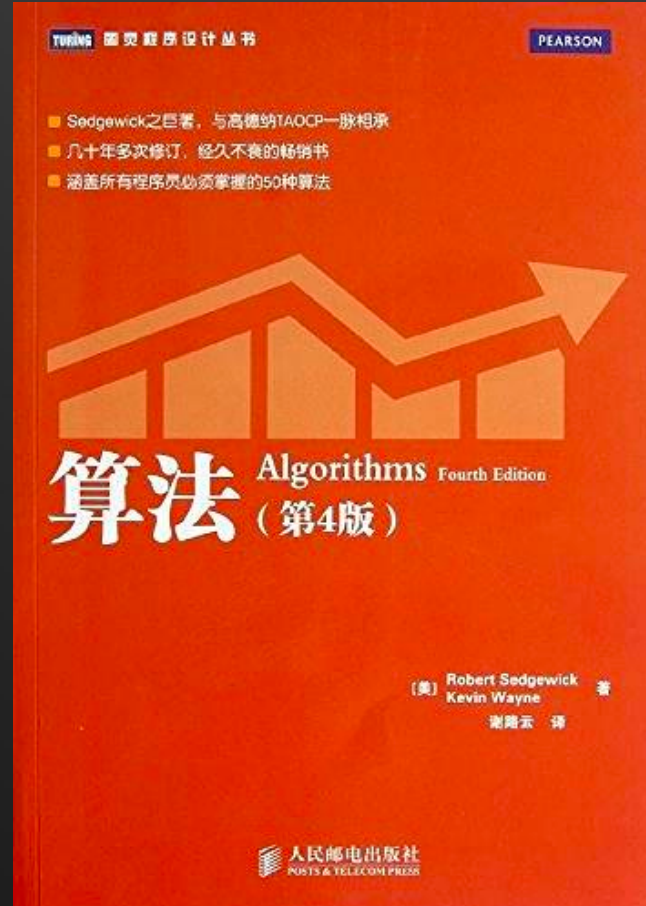
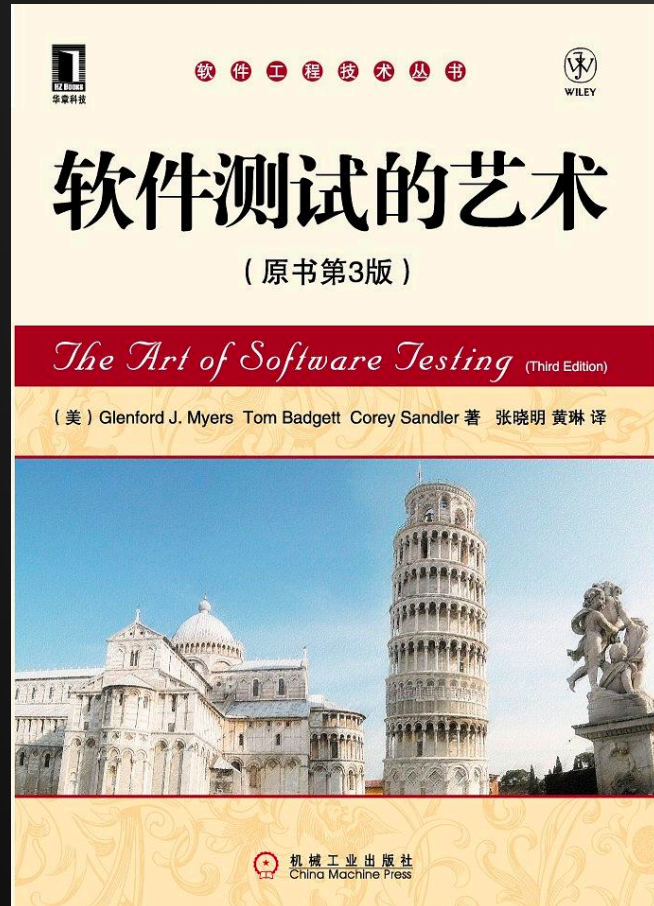
为什么要"土土"地



为什么要"土土"地



为什么要"土土"地



为什么要"土土"地

- 读过了上面的书, 我干嘛要做一个DBA ?
- 这都是 科班 学的, 咱能土点么

用一个例子介绍几种土手段

```
Replicate_Ignore_Server_Ids:  
    Master_Server_Id: 1174285442  
    Master_UUID: fdda403a-a145-11e9-9f4f-0242ac148602  
    Master_Info_File: mysql.slave_master_info  
    SQL_Delay: 0  
    SQL_Remaining_Delay: NULL  
Slave_SQL_Running_State: System lock  
    Master_Retry_Count: 1  
    Master_Bind:  
Last_IO_Error: Error:   
    Master_Bind:  
    Master_Retry_Count: 1
```

mysql system lock replication



All

Images

Videos

News

Shopping

More

Settings

Tools

About 498,000 results (0.54 seconds)

replication - MySQL slave stuck at 'System lock' most of the time ...

<https://dba.stackexchange.com/.../mysql-slave-stuck-at-system-lock-most-of-the-time-...> ▼

2 answers

Oct 24, 2014 - I had answered a question back in January 2013 concerning the thread state "**System Lock**" : What does '**system lock**' mean in **mysql** profiling a ...

mysql 5.6 gtid replication slave stuck (**system lock**)? - Database ... Nov 26, 2015

MySQL: Replication is paused with both threads running, **System** ... Jun 1, 2015

Mysql Master Slave replication performance is too low - Database ... Mar 1, 2017

MySQL 5.6 replication causes 'waiting for table lock' - Database ... Oct 10, 2014

More results from dba.stackexchange.com

More results from dba.stackexchange.com

MySQL 5.6 replication causes 'waiting for



TECHNOLOGY
ACTION
爱可生

Oct 10, 2014

MySQL Bugs: #72131: Slave running state: System lock

<https://bugs.mysql.com/bug.php?id=72131> ▼

Mar 26, 2014 - In my case the such **system lock** has been caused ROW-based **replication** with huge tables without primary key (but with indexes present).

#68460: blocked with FLUSH TABLES WITH READ **LOCK** + ... Feb 22, 2013

#87747: Open parallel copy, press a lot of ... Sep 13, 2017

#85447: Slave SQL thread **locking** issue on a certain XA ... Mar 14, 2017

#76049: Filtered **replication** events do not reset the ... Mar 4, 2015

[More results from bugs.mysql.com](#)

[More results from bugs.mysql.com](#)

[More results from bugs.mysql.com](#)



System lock on Slave - Percona Community

<https://www.percona.com/forums/questions.../mysql-and.../7861-system-lock-on-slave> ▼

Apr 6, 2013 - 3 posts - 2 authors

thread/sql/main | **System lock** | INTERNAL DDL LOG RECOVER IN ... But I found that my **replication** is still working normally. ... **MySQL Sage**.

MySQL 5.6 slave Slave_IO_State is in **system lock state** 3 posts Oct 28, 2013

mysql replication slave stuck using 100% cpu and no io ... 3 posts Nov 19, 2012

[More results from www.percona.com](#)

[More results from www.percona.com](#)

[mysql replication slave stuck using 100% cpu and no io ...](#)



用一个例子介绍几种土手段

Google 不到, 难道用百度么?



用一个例子介绍几种土手段

- gstack
- gdb --batch \
-ex "thread apply all bt" -p {pid}
- pt-pmp

用一个例子介绍几种土手段

```
Thread 2 (Thread 0x7fee102e8700 (LWP 1461)):  
#0  0x00007fee77b94945 in pthread_cond_wait@@GLIBC_2.3.2 () from /lib64/libpthread.so.0  
#1  0x0000000000ded458 in native_cond_wait (mutex=0x1e73a40 <Per_thread_connection_handler::LOCK_thread_cache>, cond=0x1  
/sb_0-26514852-1514433850.9/mysql-5.7.21/include/thr_cond.h:140  
#2  my_cond_wait (mp=0x1e73a40 <Per_thread_connection_handler::LOCK_thread_cache>, cond=0x1e73a80 <Per_thread_connection  
mysql-5.7.21/include/thr_cond.h:195  
#3  inline_mysql_cond_wait (src_line=145, that=0x1e73a80 <Per_thread_connection_handler::COND_thread_cache>, mutex=<opti  
3850.9/mysql-5.7.21/include/mysql/psi/mysql_thread.h:1184  
#4  Per_thread_connection_handler::block_until_new_connection () at /export/home/pb2/build/sb_0-26514852-1514433850.9/my  
#5  0x0000000000ded658 in handle_connection (arg=<optimized out>) at /export/home/pb2/build/sb_0-26514852-1514433850.9/m  
#6  0x0000000000f707b4 in pfs_spawn_thread (arg=0x4b98dc0) at /export/home/pb2/build/sb_0-26514852-1514433850.9/mysql-5.  
#7  0x00007fee77b90e25 in start_thread () from /lib64/libpthread.so.0  
#8  0x00007fee7664d34d in clone () from /lib64/libc.so.6
```

```
Thread 1 (Thread 0x7fee77fba740 (LWP 2399)):  
#0  0x00007fee76642a3d in poll () from /lib64/libc.so.6  
#1  0x0000000000deef9 in Mysqld_socket_listener::listen_for_connection_event (this=0x4b5a230) at /export/home/pb2/build  
#2  0x00000000007c89c4 in connection_event_loop (this=0x4b17840) at /export/home/pb2/build/sb_0-26514852-1514433850.9/my  
#3  mysqld_main (argc=96, argv=0x23815d8) at /export/home/pb2/build/sb_0-26514852-1514433850.9/mysql-5.7.21/sql/mysqld.c  
#4  0x00007fee76576c05 in __libc_start_main () from /lib64/libc.so.6  
#5  0x00000000007be479 in _start ()
```



用一个例子介绍几种土手段

Thread 1 (Thread 0x7fee77fba740 (**LWP 2399**)):



Light-weight Process

用一个例子介绍几种土手段

工具箱:

拿堆栈, 找线程, ...



用一个例子介绍几种土手段

```
frame #0: 0x0000000106bfefbda mysql`thr_multi_lock(data=0x00007fd1c0e16b18, count=0, owner=0x00007fd
frame #1: 0x000000010694ba83 mysql`mysql_lock_tables(thd=<unavailable>, tables=0x00007fd1be86cb50,
frame #2: 0x00000001069ed3ea mysql`lock_tables(thd=0x00007fd1be8d4400, tables=0x00007fd1be895620, c
frame #3: 0x00000001069eccfc mysql`open_and_lock_tables(thd=0x00007fd1be8d4400, tables=<unavailable
frame #4: 0x0000000106b7cf29 mysql`Rows_log_event::do_apply_event(Relay_log_info const*) [inlined]
frame #5: 0x0000000106b7cf0f mysql`Rows_log_event::do_apply_event(this=0x00007fd1be4347e0, rli=0x00
frame #6: 0x0000000106bbfb4f mysql`apply_event_and_update_pos(ptr_ev=0x000070000bc9a800, thd=0x0000
frame #7: 0x0000000106bb9b3a mysql`::handle_slave_sql(void *) at rpl_slave.cc:5240 [opt]
frame #8: 0x0000000106bb8bf6 mysql`::handle_slave_sql(arg=<unavailable>) at rpl_slave.cc:7377 [opt]
frame #9: 0x0000000106c5b006 mysql`::pfs_spawn_thread(arg=0x00007fd1be432b70) at pfs.cc:2190 [opt]
frame #10: 0x00007fff72b1e305 libsystem_pthread.dylib`_pthread_body + 126
```

用一个例子介绍几种土手段

```
1  * frame #0: thr_lock.c:973
2    frame #1: lock.cc:337
3    frame #2: sql_base.cc:6631
4    frame #3: sql_base.cc:6449
5    frame #4: sql_base.h:479
6    frame #5: log_event.cc:10851
7    frame #6: rpl_slave.cc:4713
8    frame #7: rpl_slave.cc:5240
9    frame #8: rpl_slave.cc:7377
10   frame #9: pfs.cc:2190 |
```

```
1  #1  sql/signal_handler.cc
2  #2  <signal handler called>
3  #3  storage/innobase/dict/dict0dict.cc
4  #4  storage/innobase/handler/ha_innodb.cc
5  #5  storage/innobase/handler/ha_innodb.cc
6  #6  storage/innobase/handler/ha_innodb.cc
7  #7  sql/handler.cc
8  #8  sql/unireg.cc
9  #9  sql/sql_table.cc
10 #10 sql/sql_table.cc
11 #11 sql/sql_table.cc
12 #12 sql/sql_parse.cc
13 #13 sql/sql_parse.cc
14 #14 sql/sql_parse.cc
15 #15 sql/sql_parse.cc
16 #16 sql/conn_handler/connection_handler_per_thread.cc
17 #17 storage/perfschema/pfs.cc
```

插曲 – unireg

In 1979, he developed an in-house database tool called UNIREG for managing databases. Since 1979, UNIREG has been rewritten in several different languages and extended to handle big databases.

No doubt the definition of big has changed for most people since then. If we take what the [MySQL 3.23 manual](#) said:

Unireg is our tty interface builder, but it uses a low level connection to our ISAM (which is used by MySQL) and because of this it is very quick. It has existed since 1979 (on Unix in C since ~1986).



It has existed since 1979 (on Unix in C since ~1986).

our ISAM (which is used by MySQL) and because of this it is very quick.



用一个例子介绍几种土手段

工具箱:

拿堆栈, 找线程, 看文件名...

用一个例子介绍几种土手段

open_and_lock_tables

```
frame #0: 0x0000000106bfefda mysql`thr_multi_lock(data=0x00007fd1c0e16b18, count=0, owner=0x00007fd
frame #1: 0x000000010694ba83 mysql`mysql_lock_tables(thd=<unavailable>, tables=0x00007fd1be86cb50,
frame #2: 0x00000001069ed3ea mysql`lock_tables(thd=0x00007fd1be8d4400, tables=0x00007fd1be895620, c
frame #3: 0x00000001069eccfc mysql`open_and_lock_tables(thd=0x00007fd1be8d4400, tables=<unavailable
frame #4: 0x0000000106b7cf29 mysql`Rows_log_event::do_apply_event(Relay_log_info const*) [inlined]
frame #5: 0x0000000106b7cf0f mysql`Rows_log_event::do_apply_event(this=0x00007fd1be4347e0, rli=0x00
frame #6: 0x0000000106bbfb4f mysql`apply_event_and_update_pos(ptr_ev=0x000070000bc9a800, thd=0x0000
frame #7: 0x0000000106bb9b3a mysql`::handle_slave_sql(void *) at rpl_slave.cc:5240 [opt]
frame #8: 0x0000000106bb8bf6 mysql`::handle_slave_sql(arg=<unavailable>) at rpl_slave.cc:7377 [opt]
frame #9: 0x0000000106c5b006 mysql`::pfs_spawn_thread(arg=0x00007fd1be432b70) at pfs.cc:2190 [opt]
frame #10: 0x00007fff72b1e305 libsystem_pthread.dylib`_pthread_body + 126
```

用一个例子介绍几种土手段

找入口:

- 抓堆栈
- gdb断点 常用入口:
 - handle_slave_sql
 - mysql_execute_command
- dtrace:
 - query_parse_*/ select_* / net_read_* / handler_rdlock_*

用一个例子介绍几种土手段

工具箱:

拿堆栈, 找线程, 看文件名, 找入口...

用一个例子介绍几种土手段

open_and_lock_tables

```
frame #0: 0x0000000106bfefda mysql`thr_multi_lock(data=0x00007fd1c0e16b18, count=0, owner=0x00007fd
frame #1: 0x000000010694ba83 mysql`mysql_lock_tables(thd=<unavailable>, tables=0x00007fd1be86cb50,
frame #2: 0x00000001069ed3ea mysql`lock_tables(thd=0x00007fd1be8d4400, tables=0x00007fd1be895620, c
frame #3: 0x00000001069eccfc mysql`open_and_lock_tables(thd=0x00007fd1be8d4400, tables=<unavailable
frame #4: 0x0000000106b7cf29 mysql`Rows_log_event::do_apply_event(Relay_log_info const*) [inlined]
frame #5: 0x0000000106b7cf0f mysql`Rows_log_event::do_apply_event(this=0x00007fd1be4347e0, rli=0x00
frame #6: 0x0000000106bbfb4f mysql`apply_event_and_update_pos(ptr_ev=0x000070000bc9a800, thd=0x0000
frame #7: 0x0000000106bb9b3a mysql`::handle_slave_sql(void *) at rpl_slave.cc:5240 [opt]
frame #8: 0x0000000106bb8bf6 mysql`::handle_slave_sql(arg=<unavailable>) at rpl_slave.cc:7377 [opt]
frame #9: 0x0000000106c5b006 mysql`::pfs_spawn_thread(arg=0x00007fd1be432b70) at pfs.cc:2190 [opt]
frame #10: 0x00007fff72b1e305 libsystem_pthread.dylib`_pthread_body + 126
```

用一个例子介绍几种土手段

```
10848      /* A small test to verify that objects have consistent types */
10849      DEBUG_ASSERT(sizeof(thd->variables.option_bits) == sizeof(OPTION_RELAXED_UNIQUE_CHECKS));
10850
10851      if (open_and_lock_tables(thd, rli->tables_to_lock, 0))
10852      {
10853          uint actual_error= thd->get_stmt_da()->mysql_errno();
10854          if (thd->is_slave_error || thd->is_fatal_error)
10855          {
10856              if (ignored_error_code(actual_error))
10857              {
10858                  if (log_warnings > 1)
10859                      rli->report(WARNING_LEVEL, actual_error,
10860                                  "Error executing row event: '%s'",
10861                                  (actual_error ? thd->get_stmt_da()->message_text() :
10862                                  "unexpected success or fatal error"));
```



ACTION
爱可生

用一个例子介绍几种土手段

这都是些啥



用一个例子介绍几种土手段

```
/*  
    .....  
*/  
if (!thd->lock)  
{  
    ...  
    if (open_and_lock_tables(...))  
    {  
        ...  
    }  
    ...  
}
```

用一个例子介绍几种土手段

```
/* If there is no locks taken, this  
is the first binrow event seen after  
the table map events.
```

```
We should then lock all the tables  
used in the transaction and proceed  
with execution of the actual event. */
```

用一个例子介绍几种土手段

工具箱:

拿堆栈, 找线程, 看文件名, 找入口,
看结构, 读注释, ...

用一个例子介绍几种土手段

为什么要锁全部表？



用一个例子介绍几种土手段

```
git log -u -G ".*lock_tables.*" \  
sql/log_event.cc
```

commit 22eda18cec3bc84d724553fb28ef1e6eb9d3ef5c

Author: Roy Lyseng <roy.lyseng@oracle.com>

Date: Fri Jan 30 17:23:53 2015 +0100

WL#5275: Process subqueries in FROM clause in the same way as view

Complete implementation.

...

commit 694431f4223e60a916d5a290709b2bbd0f8ce1ac

Author: Jon Olav Hauglid <jon.hauglid@oracle.com>

Date: Tue Jan 27 17:22:03 2015 +0100

Bug#20435863: FIX #INCLUDE DEPENDENCIES IN RPL HEADERS

Part 1: More cleanup of #includes. Make more self-contained,
remove unnecessary dependencies.



用一个例子介绍几种土手段

找到open_and_lock_tables

首次出现的位置



用一个例子介绍几种土手段

```
diff --git a/sql/log_event.cc b/sql/log_event.cc
index b0deee62fd7..b0437b7c2e5 100644
--- a/sql/log_event.cc
+++ b/sql/log_event.cc
@@ -7480,7 +7480,7 @@ int Rows_log_event::do_apply_event(Relay_log_info const *rli)
    /* A small test to verify that objects have consistent types */
    DEBUG_ASSERT(sizeof(thd->variables.option_bits) == sizeof(OPTION_RELAXED_UNIQUE_CHECKS));

-   if (simple_open_n_lock_tables(thd, rli->tables_to_lock))
+   if (open_and_lock_tables(thd, rli->tables_to_lock, FALSE, 0))
    {
        uint actual_error= thd->stmt_da->sql_errno();
        if (thd->is_slave_error || thd->is_fatal_error)
```

用一个例子介绍几种土手段

```
commit dd42aab8406e70caec4f58435c0281cddc8ea2c0
```

```
Author: Jon Olav Hauglid <jon.hauglid@sun.com>
```

```
Date: Wed Feb 24 18:04:00 2010 +0100
```

Followup to [Bug#45225 Locking: hang if drop table with no timeout](#)

...

The patch [also simplifies the open tables API](#). ... Now, this convenience function is

gone and the main function is [renamed](#) to `open_and_lock_tables()`



用一个例子介绍几种土手段

找到 `simple_open_n_lock_tables`

首次出现的位置



用一个例子介绍几种土手段

```
@@ -6452,72 +6466,23 @@ int Rows_log_event::do_apply_event(Relay_log_info const *rli)
/* A small test to verify that objects have consistent types */
DEBUG_ASSERT(sizeof(thd->options) == sizeof(OPTION_RELAXED_UNIQUE_CHECKS));

-
- while ((error= lock_tables(thd, rli->tables_to_lock,
-                             rli->tables_to_lock_count, &need_reopen)))
+ if (simple_open_n_lock_tables(thd, rli->tables_to_lock))
{
```

commit 812fa8b24c6c8cec1209409d1b18c122ae2b776b

Author: mats@mats-laptop.(none) <>

Date: Mon May 12 19:50:53 2008 +0200

BUG#36197: flush tables (or little table cache) can cause crash on slave

When flushing tables, there were a slight chance that the flush was occurring **between processing of two table map events.**

Since the tables are opened one by one, it might result in that the tables were not valid and that sub-sequent locking of tables would cause the slave to crash.



用一个例子介绍几种土手段

The problem is solved by opening and locking all tables at once using `simple_open_n_lock_tables()`.

用一个例子介绍几种土手段

找到 lock_tables

首次出现的位置



commit ad126d90e019f223470e73e1b2b528f9007c4532

Author: lars@mysql.com <>

Date: Thu Dec 22 06:39:02 2005 +0100

WL#1012: All changes as one single
changeset.

This includes both code and test cases.



用一个例子介绍几种土手段

插播 腾讯贡献的 Work Log
Instant Add Column



默认值: abc



Affects: Server-8.0 — Status: Complete

Description

Requirements

High Level Architecture

Low Level Design

Porting Tecent's patch on Instant Add Column.

To summarize the idea:

1. It introduced a new info bits on the record to signify the row has new rows added instantly
2. If the bit is set, then there will be one or two additional bytes in the row header to signify the extra columns added
3. A new system table is added to record the "default value" for each added column
4. The rows before instant add column would always have the old number of columns for its life time (until deleted). Such number of "core" rows will be stored in `se_private_data` of the `mysql.tables`
5. For "old" rows, the default value will be looked up from the new system tables and appended before return to server.

High-Level Design

- Problems of current ADD COLUMN
- Operations which can be instant
- Algorithm
- Row format compatibility
- Integration with DD
- DMLs

High-Level Design

- Rollback
- Recovery
- Replication
- IMPORT/EXPORT
- Displayed in I_S.innodb*
- Side effects

Side effects

a) ... not fix a corrupted index

b) ...some rows may in fact too big for the table, ..., it would be detected when the record gets updated.

用一个例子介绍几种土手段

工具箱:

拿堆栈, 找线程, 看文件名, 找入口,
看结构, 读注释, 找历史,
查提交, 读WL...

commit ad126d90e019f223470e73e1b2b528f9007c4532

Author: lars@mysql.com <>

Date: Thu Dec 22 06:39:02 2005 +0100

WL#1012: All changes as one single
changeset.

This includes both code and test cases.



[Forums](#)[Bugs](#)[Worklog](#)[Labs](#)[Planet MySQL](#)[News and Events](#)[Community](#)**Unable to Locate Task**

CLOSE ✕

Unfortunately, either a task with that ID does not exist, or it is not a publicly available Worklog entry.

Search Tasks

Worklog tasks

Worklog #: 1012

Go

Text: instant

历史没有真相，只残存一个道理。

—— 马未都



用一个例子介绍几种土手段

- RBR 要在 TABLE_MAP_EVENT 后第一个Event进行锁, 锁所有相关的表
- 这些锁必须 原子完成, 否则插入flush操作会导致崩溃
- 查WL可以找到设计考量 (可惜 变更太旧, 无从追溯)

彩蛋

MySQL xx功能是否靠谱



彩蛋

用例: `mysql-test/suite/rpl/t/*.test`

结果: `mysql-test/suite/rpl/r/*.result`

彩蛋

测试 replication-ignore-db 的例子

```
1 # see if --replicate-ignore-db works
2
3 --source include/not_group_replication_plugin.inc
4 --source include/not_gtid_enabled.inc
5 --source include/master-slave.inc
6
7 --disable_warnings
8 drop database if exists mysqltest1;
9 drop database if exists mysqltest2;
10 --enable_warnings
11 create database mysqltest1;
12 create database mysqltest2;
13
14 use mysqltest1;
15 create table t1 (a int);
16 insert into t1 values(1);
17 --source include/sync_slave_sql_with_master.inc
18 --error 1146
19 select * from mysqltest1.t1;
20
21 connection master;
22 use mysqltest2;
23 create table t1 (a int);
24 insert into t1 values(1);
25 --source include/sync_slave_sql_with_master.inc
26 select * from mysqltest2.t1;
27
28 # cleanup
29 connection master;
30 drop database mysqltest1;
31 drop database mysqltest2;
32 --source include/sync_slave_sql_with_master.inc
33 --source include/rpl_end.inc
34
```

环境搭建

测试

清理

环境搭建

```
1  # see if --replicate-ignore-db works
2
3  --source include/not_group_replication_plugin.inc
4  --source include/not_gtid_enabled.inc
5  --source include/master-slave.inc
6
7  --disable_warnings
8  drop database if exists mysqltest1;
9  drop database if exists mysqltest2;
10 --enable_warnings
11 create database mysqltest1;
12 create database mysqltest2;
```

测试

```
14 use mysqltest1;
15 create table t1 (a int);
16 insert into t1 values(1);
17 --source include/sync_slave_sql_with_master.inc
18 --error 1146
19 select * from mysqltest1.t1;
20
21 connection master;
22 use mysqltest2;
23 create table t1 (a int);
24 insert into t1 values(1);
25 --source include/sync_slave_sql_with_master.inc
26 select * from mysqltest2.t1;
27
```

清理

```
28 # cleanup
29 connection master;
30 drop database mysqltest1;
31 drop database mysqltest2;
32 --source include/sync_slave_sql_with_master.inc
33 --source include/rpl_end.inc
```

[tachikoma@miao] [19:09:32] [~/Code/mysql-server/mysql-test/suite/rpl/t15.7]

→ ls rpl_row*.test

rpl_row_001.test	rpl_row_hash_scan.test	rpl_row_rollback_to_savepoint.test
rpl_row_4_bytes.test	rpl_row_hash_scan_sanity.test	rpl_row_slave_skip_error_all.test
rpl_row_NOW.test	rpl_row_idempotency.test	rpl_row_sp001.test
rpl_row_USER.test	rpl_row_ignorable_event.test	rpl_row_sp002_innodb.test
rpl_row_UUID.test	rpl_row_img_blobs.test	rpl_row_sp003.test
rpl_row_basic_11bugs.test	rpl_row_img_eng_full.test	rpl_row_sp005.test
rpl_row_basic_2myisam.test	rpl_row_img_eng_min.test	rpl_row_sp006_InnoDB.test
rpl_row_basic_3innodb.test	rpl_row_img_eng_noblob.test	rpl_row_sp007_innodb.test
rpl_row_basic_8partition.test	rpl_row_img_idx_full.test	rpl_row_sp008.test
rpl_row_basic_allow_batching.test	rpl_row_img_idx_min.test	rpl_row_sp009.test
rpl_row_binlog_max_cache_size.test	rpl_row_img_idx_noblob.test	rpl_row_sp010.test
rpl_row_blob_innodb.test	rpl_row_img_misc.test	rpl_row_sp011.test
rpl_row_blob_myisam.test	rpl_row_img_sanity.test	rpl_row_sp012.test
rpl_row_colSize.test	rpl_row_implicit_commit_binlog.test	rpl_row_tabledefs_2myisam.test
rpl_row_conflicts.test	rpl_row_inexist_tbl.test	rpl_row_tabledefs_3innodb.test
rpl_row_corrupt.test	rpl_row_lcase_tblnames.test	rpl_row_tbl_metadata.test
rpl_row_corruption.test	rpl_row_loaddata_concurrent.test	rpl_row_trig001.test
rpl_row_crash_safe.test	rpl_row_log.test	rpl_row_trig002.test
rpl_row_create_select.test	rpl_row_log_innodb.test	rpl_row_trig003.test
rpl_row_create_table.test	rpl_row_max_relay_size.test	rpl_row_trig004.test
rpl_row_drop.test	rpl_row_merge_engine.test	rpl_row_trunc_temp.test
rpl_row_drop_create_temp_table.test	rpl_row_mixing_engines.test	rpl_row_unsafe_funcs.test
rpl_row_event_max_size.test	rpl_row_mts_crash_safe.test	rpl_row_until.test
rpl_row_find_row.test	rpl_row_mts_rec_crash_safe.test	rpl_row_utf16.test
rpl_row_flush_tbls.test	rpl_row_mysqlbinlog.test	rpl_row_utf32.test
rpl_row_func001.test	rpl_row_rec_comp_innodb.test	rpl_row_view01.test
rpl_row_func002.test	rpl_row_rec_comp_myisam.test	rpl_row_wide_table.test
rpl_row_func003.test	rpl_row_record_find_myisam.test	
rpl_row_gtid_disconnect_drop_temporary_table.test	rpl_row_reset_slave.test	

工具箱:

拿堆栈, 找线程, 看文件名, 找入口,
看结构, 读注释, 找历史,
查提交, 读WL, 看用例, ...

